

a) Descrição do Problema

Diagnóstico Inteligente de Falhas em Sistemas de Distribuição Elétrica com LLMs

Sistemas de distribuição elétrica sofrem falhas complexas (curtos-circuitos, sobretensão, desequilíbrio de fases, sobrecarga) que exigem diagnóstico rápido e preciso.

Tradicionalmente, engenheiros analisam manualmente alarmes e sinais — processo lento e sujeito a erro humano.

A solução usa um LLM (Claude) para:

1. Receber leituras de sensores em tempo real (tensão, corrente, fator de potência, temperatura)
2. Diagnosticar o tipo de falha e sua severidade
3. Recomendar ações corretivas priorizadas
4. Gerar um laudo técnico estruturado

Código:

Simula leituras elétricas

(tensão, corrente, temperatura, fator de potência etc.)

Compara os valores com regras de engenharia

Exemplo:

- corrente muito alta → curto-circuito
- tensão alta → sobretensão
- FP baixo → problema de fator de potência

Gera automaticamente o diagnóstico

com:

- tipo de falha
- severidade
- ações recomendadas
- laudo técnico

(b) código fonte python da solução do problema;

(c) representações visuais da solução

```
# =====  
# DIAGNÓSTICO INTELIGENTE DE FALHAS ELÉTRICAS  
# VERSÃO 100% GRATUITA - SEM API - GOOGLE COLAB  
# =====  
  
# =====
```

```

# 1. INSTALAR DEPENDÊNCIAS
# =====

!pip install -q pandas matplotlib seaborn numpy

# =====
# 2. IMPORTAÇÕES
# =====

import json
import random
import time

from datetime import datetime
from dataclasses import dataclass, asdict

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

# =====
# 3. CONFIGURAÇÃO DOS GRÁFICOS
# =====

%matplotlib inline

sns.set_style("whitegrid")

# =====
# 4. ESTRUTURAS DE DADOS
# =====

@dataclass
class LeituraSensor:

    timestamp: str
    barramento: str

    tensao_Va: float
    tensao_Vb: float
    tensao_Vc: float

```

```

    corrente_Ia: float
    corrente_Ib: float
    corrente_Ic: float

    fator_potencia: float
    temperatura_C: float
    frequencia_Hz: float

    tensao_nominal: float
    corrente_nominal: float

@dataclass
class DiagnosticoFalha:

    barramento: str
    timestamp: str

    tipo_falha: str
    severidade: str
    severidade_score: int

    causa_provavel: str

    acoes_imediatas: list
    acoes_preventivas: list

    tempo_resposta_recomendado: str
    laudo_tecnico: str

# =====
# 5. SIMULADOR DA REDE ELÉTRICA
# =====

class SimuladorRedeEletrica:

    CENARIOS = {

        "normal": {
            "descricao": "Operação normal",
            "fp": 0.97,
            "temp": 45

```

```

    },

    "sobrecarga": {
        "descricao": "Sobrecarga elétrica",
        "fp": 0.86,
        "temp": 82
    },

    "curto_circuito": {
        "descricao": "Curto-circuito fase-terra",
        "fp": 0.40,
        "temp": 110
    },

    "desequilibrio": {
        "descricao": "Desequilíbrio entre fases",
        "fp": 0.91,
        "temp": 66
    },

    "sobretensao": {
        "descricao": "Sobretensão",
        "fp": 0.95,
        "temp": 58
    },

    "baixo_fp": {
        "descricao": "Baixo fator de potência",
        "fp": 0.71,
        "temp": 60
    }
}

def __init__(self):

    self.Vn = 13800
    self.In = 400

def gerar(self, barramento, cenario):

    ruído = lambda x=0.02: random.gauss(0, x)

    if cenario == "normal":

```

```

Va = self.Vn * (1 + ruido(0.01))
Vb = self.Vn * (1 + ruido(0.01))
Vc = self.Vn * (1 + ruido(0.01))

Ia = self.In * 0.75
Ib = self.In * 0.74
Ic = self.In * 0.76

elif cenario == "sobrecarga":

    Va = self.Vn * 0.92
    Vb = self.Vn * 0.91
    Vc = self.Vn * 0.90

    Ia = self.In * 1.45
    Ib = self.In * 1.50
    Ic = self.In * 1.42

elif cenario == "curto_circuito":

    Va = self.Vn * 0.10
    Vb = self.Vn * 1.02
    Vc = self.Vn * 1.01

    Ia = self.In * 8.5
    Ib = self.In * 0.72
    Ic = self.In * 0.70

elif cenario == "desequilibrio":

    Va = self.Vn * 1.00
    Vb = self.Vn * 0.87
    Vc = self.Vn * 1.12

    Ia = self.In * 0.82
    Ib = self.In * 1.18
    Ic = self.In * 0.63

elif cenario == "sobretensao":

    Va = self.Vn * 1.24
    Vb = self.Vn * 1.23

```

```

        Vc = self.Vn * 1.25

        Ia = self.In * 0.80
        Ib = self.In * 0.82
        Ic = self.In * 0.81

    elif cenario == "baixo_fp":

        Va = self.Vn * 1.01
        Vb = self.Vn * 1.00
        Vc = self.Vn * 1.02

        Ia = self.In * 0.95
        Ib = self.In * 0.96
        Ic = self.In * 0.94

    c = self.CENARIOS[cenario]

    return LeituraSensor(

        timestamp=datetime.now().isoformat(),

        barramento=barramento,

        tensao_Va=round(Va,1),
        tensao_Vb=round(Vb,1),
        tensao_Vc=round(Vc,1),

        corrente_Ia=round(Ia,2),
        corrente_Ib=round(Ib,2),
        corrente_Ic=round(Ic,2),

        fator_potencia=c["fp"],

        temperatura_C=c["temp"],

        frequencia_Hz=60.0,

        tensao_nominal=self.Vn,
        corrente_nominal=self.In
    )

```

```

# =====

```

```
# 6. MOTOR DE DIAGNÓSTICO INTELIGENTE
```

```
# =====
```

```
class MotorDiagnostico:
```

```
    def diagnosticar(self, leitura):
```

```
        Vn = leitura.tensao_nominal
```

```
        In = leitura.corrente_nominal
```

```
        media_tensao = (  
            leitura.tensao_Va +  
            leitura.tensao_Vb +  
            leitura.tensao_Vc  
        ) / 3
```

```
        desvio_tensao = max(  
  
            abs(leitura.tensao_Va - media_tensao),  
            abs(leitura.tensao_Vb - media_tensao),  
            abs(leitura.tensao_Vc - media_tensao)  
  
        ) / media_tensao * 100
```

```
        corrente_max = max(  
  
            leitura.corrente_Ia,  
            leitura.corrente_Ib,  
            leitura.corrente_Ic  
        )
```

```
        corrente_pct = corrente_max / In * 100
```

```
        # =====  
        # CURTO-CIRCUITO  
        # =====
```

```
        if corrente_pct > 500:
```

```
            return DiagnosticoFalha(  
  
                barramento=leitura.barramento,  
                timestamp=leitura.timestamp,
```

```

        tipo_falha="Curto-circuito monofásico",

        severidade="EMERGÊNCIA",

        severidade_score=100,

        causa_provavel=
        "Corrente extremamente elevada indicando falta
fase-terra.",

        acoes_imediatas=[
            "Abrir disjuntor imediatamente",
            "Isolar trecho afetado",
            "Acionar equipe de emergência"
        ],

        acoes_preventivas=[
            "Revisar coordenação da proteção",
            "Inspeccionar isoladores"
        ],

        tempo_resposta_recomendado("< 1 minuto",

        laudo_tecnico=
        "Foi detectada corrente de falta extremamente elevada "
        "com afundamento severo de tensão na fase A."
    )

# =====
# SOBRECARGA
# =====

elif corrente_pct > 130:

    return DiagnosticoFalha(

        barramento=leitura.barramento,
        timestamp=leitura.timestamp,

        tipo_falha="Sobrecarga elétrica",

        severidade="CRÍTICO",

```



```

        severidade_score=85,

        causa_provavel=
            "Carga acima da capacidade nominal do circuito.",

        acoes_imediatas=[
            "Redistribuir cargas",
            "Reduzir demanda",
            "Monitorar temperatura"
        ],

        acoes_preventivas=[
            "Revisar dimensionamento",
            "Expandir capacidade"
        ],

        tempo_resposta_recomendado="até 5 minutos",

        laudo_tecnico=
            "A corrente excede significativamente a nominal, "
            "indicando sobrecarga sustentada."
    )

# =====
# SOBRETENSÃO
# =====

elif leitura.tensao_Va > Vn * 1.15:

    return DiagnosticoFalha(

        barramento=leitura.barramento,
        timestamp=leitura.timestamp,

        tipo_falha="Sobretensão",

        severidade="CRÍTICO",

        severidade_score=80,

        causa_provavel=
            "Elevação anormal da tensão da rede.",

```

```

        acoes_imediatas=[
            "Verificar reguladores",
            "Inspeccionar transformadores"
        ],

        acoes_preventivas=[
            "Ajustar taps",
            "Melhorar regulação"
        ],

        tempo_resposta_recomendado="até 10 minutos",

        laudo_tecnico=
            "As tensões estão acima do limite operacional
permitido."
    )

# =====
# DESEQUILÍBRIO
# =====

elif desvio_tensao > 8:

    return DiagnosticoFalha(

        barramento=leitura.barramento,
        timestamp=leitura.timestamp,

        tipo_falha="Desequilíbrio entre fases",

        severidade="ATENÇÃO",

        severidade_score=60,

        causa_provavel=
            "Distribuição desigual de carga entre as fases.",

        acoes_imediatas=[
            "Balancear cargas",
            "Monitorar correntes"
        ],

```

```

        acoes_preventivas=[
            "Revisar distribuição das cargas"
        ],

        tempo_resposta_recomendado="até 1 hora",

        laudo_tecnico=
            "Foi identificado desequilíbrio significativo "
            "de tensão e corrente entre fases."
    )

# =====
# BAIXO FP
# =====

elif leitura.fator_potencia < 0.85:

    return DiagnosticoFalha(

        barramento=leitura.barramento,
        timestamp=leitura.timestamp,

        tipo_falha="Baixo fator de potência",

        severidade="ATENÇÃO",

        severidade_score=55,

        causa_provavel=
            "Excesso de carga reativa indutiva.",

        acoes_imediatas=[
            "Acionar banco de capacitores"
        ],

        acoes_preventivas=[
            "Automatizar correção de FP"
        ],

        tempo_resposta_recomendado="até 24 horas",

        laudo_tecnico=

```

```

        "O fator de potência está abaixo do limite
recomendado."
    )

    # =====
    # NORMAL
    # =====

else:

    return DiagnosticoFalha(

        barramento=leitura.barramento,
        timestamp=leitura.timestamp,

        tipo_falha="Operação normal",

        severidade="NORMAL",

        severidade_score=10,

        causa_provavel=
        "Sistema operando dentro da normalidade.",

        acoes_imediatas=[
            "Nenhuma ação necessária"
        ],

        acoes_preventivas=[
            "Continuar monitoramento"
        ],

        tempo_resposta_recomendado=
        "próxima manutenção",

        laudo_tecnico=
        "Todos os parâmetros elétricos encontram-se "
        "dentro dos limites operacionais."
    )

# =====
# 7. VISUALIZAÇÃO
# =====

```

```

class Visualizador:

    CORES = {

        "NORMAL": "#27ae60",
        "ATENÇÃO": "#f39c12",
        "CRÍTICO": "#e74c3c",
        "EMERGÊNCIA": "#8e44ad"
    }

    def __init__(self, leituras, diagnosticos):

        self.leituras = leituras
        self.diagnosticos = diagnosticos

    def gerar(self):

        fig, axs = plt.subplots(2, 2, figsize=(16, 10))

        barramentos = [l.barramento for l in self.leituras]

        # =====
        # TENSÕES
        # =====

        Va = [l.tensao_Va for l in self.leituras]
        Vb = [l.tensao_Vb for l in self.leituras]
        Vc = [l.tensao_Vc for l in self.leituras]

        x = np.arange(len(barramentos))

        axs[0,0].bar(x-0.2, Va, width=0.2, label="Va")
        axs[0,0].bar(x, Vb, width=0.2, label="Vb")
        axs[0,0].bar(x+0.2, Vc, width=0.2, label="Vc")

        axs[0,0].set_title("Tensões")
        axs[0,0].set_xticks(x)
        axs[0,0].set_xticklabels(barramentos)

        axs[0,0].legend()

        # =====

```

```

# CORRENTES
# =====

Ia = [l.corrente_Ia for l in self.leituras]
Ib = [l.corrente_Ib for l in self.leituras]
Ic = [l.corrente_Ic for l in self.leituras]

axs[0,1].bar(x-0.2, Ia, width=0.2, label="Ia")
axs[0,1].bar(x, Ib, width=0.2, label="Ib")
axs[0,1].bar(x+0.2, Ic, width=0.2, label="Ic")

axs[0,1].set_title("Correntes")
axs[0,1].set_xticks(x)
axs[0,1].set_xticklabels(barramentos)

axs[0,1].legend()

# =====
# SCORE
# =====

scores = [
    d.severidade_score
    for d in self.diagnosticos
]

axs[1,0].bar(

    barramentos,
    scores,

    color=[
        self.CORES[d.severidade]
        for d in self.diagnosticos
    ]
)

axs[1,0].set_title("Score de Severidade")
axs[1,0].set_ylim(0,100)

# =====
# FP X TEMPERATURA
# =====

```

```

        fps = [l.fator_potencia for l in self.leituras]

        temps = [l.temperatura_C for l in self.leituras]

        axs[1,1].scatter(

            fps,
            temps,

            s=220,

            c=[
                self.CORES[d.severidade]
                for d in self.diagnosticos
            ]
        )

        axs[1,1].set_title("FP x Temperatura")

        axs[1,1].set_xlabel("Fator de Potência")

        axs[1,1].set_ylabel("Temperatura")

        plt.tight_layout()

        plt.savefig(
            "diagnostico_eletrico.png",
            dpi=150
        )

        plt.show()

# =====
# 8. EXECUÇÃO PRINCIPAL
# =====

def main():

    print("=" * 60)
    print("DIAGNÓSTICO INTELIGENTE DE FALHAS ELÉTRICAS")
    print("=" * 60)

```

```

cenarios = [

    ("BUS-01", "normal"),
    ("BUS-02", "sobrecarga"),
    ("BUS-03", "curto_circuito"),
    ("BUS-04", "desequilibrio"),
    ("BUS-05", "sobretensao"),
    ("BUS-06", "baixo_fp"),
]

simulador = SimuladorRedeEletrica()

motor = MotorDiagnostico()

leituras = []
diagnosticos = []

print("\n📡 Executando diagnósticos...\n")

for barramento, cenario in cenarios:

    print(f"[{barramento}] {cenario}")

    leitura = simulador.gerar(
        barramento,
        cenario
    )

    diagnostico = motor.diagnosticar(leitura)

    leituras.append(leitura)

    diagnosticos.append(diagnostico)

    print(f"✅ {diagnostico.severidade}")
    print(f"⚡ {diagnostico.tipo_falha}\n")

    time.sleep(0.5)

# =====
# RELATÓRIO
# =====

```



```

print("\n" + "=" * 60)
print("RELATÓRIO FINAL")
print("=" * 60)

for d in diagnosticos:

    print("\n-----")

    print(f"Barramento : {d.barramento}")

    print(f"Falha      : {d.tipo_falha}")

    print(f"Severidade : {d.severidade}")

    print(f"Score      : {d.severidade_score}")

    print(f"\nCausa:")
    print(d.causa_provavel)

    print(f"\nLaudo:")
    print(d.laudo_tecnico)

# =====
# EXPORTAR JSON
# =====

with open(

    "diagnosticos_exportados.json",
    "w",
    encoding="utf-8"

) as f:

    json.dump(

        [asdict(d) for d in diagnosticos],

        f,

        ensure_ascii=False,
        indent=2

    )

```

```

print("\n💾 JSON exportado!")

# =====
# GRÁFICOS
# =====

print("\n🎨 Gerando gráficos...")

viz = Visualizador(
    leituras,
    diagnosticos
)

viz.gerar()

print("\n✅ EXECUÇÃO CONCLUÍDA!")

# =====
# 9. EXECUTAR
# =====

main()

# =====
# 10. DOWNLOAD AUTOMÁTICO DOS ARQUIVOS
# =====

from google.colab import files

files.download("diagnostico_eletrico.png")

files.download("diagnosticos_exportados.json")

```